3A: Sparse Autoencoder Implementation

1. **Obstacle: Matrix Dimensionality and Vectorization**
   The Problem: One of the biggest challenges was ensuring that the weight matrices and the data patches were aligned correctly for matrix multiplication. For example, multiplying the input data of shape (10000, 64) by the first weight matrix (64, 25) to get the hidden layer activations required a solid grasp of how np.dot() handles multidimensional arrays.
   The Solution: The .shape attribute was used frequently during debugging to print out the dimensions of variables at each step of the forward and backward passes. Drawing out the network layers on paper helped visualize how the 64 input pixels mapped to the 25 hidden neurons.
2. **Obstacle: Numerical Stability (NaN Errors)**
   The Problem: While implementing the KL Divergence for the sparsity penalty, RuntimeWarnings regarding "divide by zero" or "invalid values in log" occurred. This happened because the average activation would occasionally become exactly 0 or 1, making the log calculation impossible.
   The Solution: A tiny "epsilon" value (10^-5) was introduced to the logarithmic terms. By calculating log(rho + epsilon), the code never attempted to calculate the logarithm of zero, which stabilized the training process and allowed the cost function to converge smoothly.
3. **Obstacle: Data Normalization for Sigmoid**
   The Problem: Initially, the autoencoder was not learning "edges" and was instead producing blurry, gray blobs. This was because the raw pixel values were not optimized for the sigmoid activation function, which is most sensitive to changes near the center of its range.
   The Solution: The normalizeData function was implemented to shift the pixel values from their original range into a tighter [0.1, 0.9] range. This ensures that the gradients remain strong enough for the network to learn efficiently, eventually resulting in the sharp "Gabor filter" edge detectors required for the assignment.

3B: Shakespearean Text Generation

1. **Model Description**
   The model implemented for this assignment is a decoder-only Transformer architecture, similar to the GPT-2 model.

   Model Parameters:

   n_layer = 6: This depth allows the model to learn complex hierarchical relationships in text while remaining small enough to train on a standard laptop in under 20 minutes.

   n_head = 6 and n_embd = 384: By setting the embedding dimension to 384 and using 6 heads, an attention dimension of 64 per head (384 / 6) is created. This balanced configuration allows each head to focus on different linguistic features simultaneously.

   context_length = 256: This window provides enough "memory" for the model to maintain the context of a short conversation or a monologue in Shakespearean style.

dropout = 0.2: This fraction was chosen to prevent overfitting. Since the Shakespeare dataset is relatively small, a higher dropout ensures the model does not memorize the text word-for-word.

2. **Model Evaluation**
   The model consists of 10,690,625 trainable parameters. Evaluation shows that the model effectively captured the statistical distribution of the Shakespeare dataset.

   Evidence of Learning:

   Structural Formatting: The generated text successfully follows the script format of a play, correctly using character names in all-caps followed by colons.

   Vocabulary: The output contains character names specific to the training data, such as "Mistress Overdone" and "Provost," indicating the model learned the specific world of the data.

   Linguistic Style: The model uses period-accurate words such as "sovereign," "substance," and "nay," demonstrating that the self-attention mechanism successfully associated these tokens with the broader Shakespearean style.

   Output:

   ROMEO:

   To the word is such a substance: unto the consent

   the father of the new against the people that I

   shall perceive their sovereign. This youth is

   butchering for the party.

   MISTRESS OVERDONE:

   What draw goes of shameful bow?

   Provost:

   Nay, but you will

3. **Reflections**
   A major takeaway from this lab was the importance of Causal Masking. Without the lower-triangular mask in the attention block, the model would "cheat" during training by looking at future tokens it is supposed to be predicting. Additionally, because Transformers process data in parallel, Positional Encodings are the only thing that gives the model a sense of word order. Without them, the output would be a scrambled "bag of words."

3C: Denoising Diffusion Probabilistic Models (DDPM)

1. **Model Description and Key Components**

A Denoising Diffusion Probabilistic Model (DDPM) is a generative AI architecture that learns to create data by reversing a gradual noise-addition process.

The Forward Process (Noise Scheduler): This is a fixed mathematical process that gradually corrupts an image with Gaussian noise over a series of timesteps (T = 1000). By using the reparameterization trick, the NoiseScheduler can jump to any arbitrary timestep t and apply the exact amount of noise required.

The Reverse Process (U-Net Architecture): This is the actual neural network that learns to reverse the forward process. Instead of predicting the clean image directly, the U-Net is trained to predict the noise that was added to the image at a specific timestep.

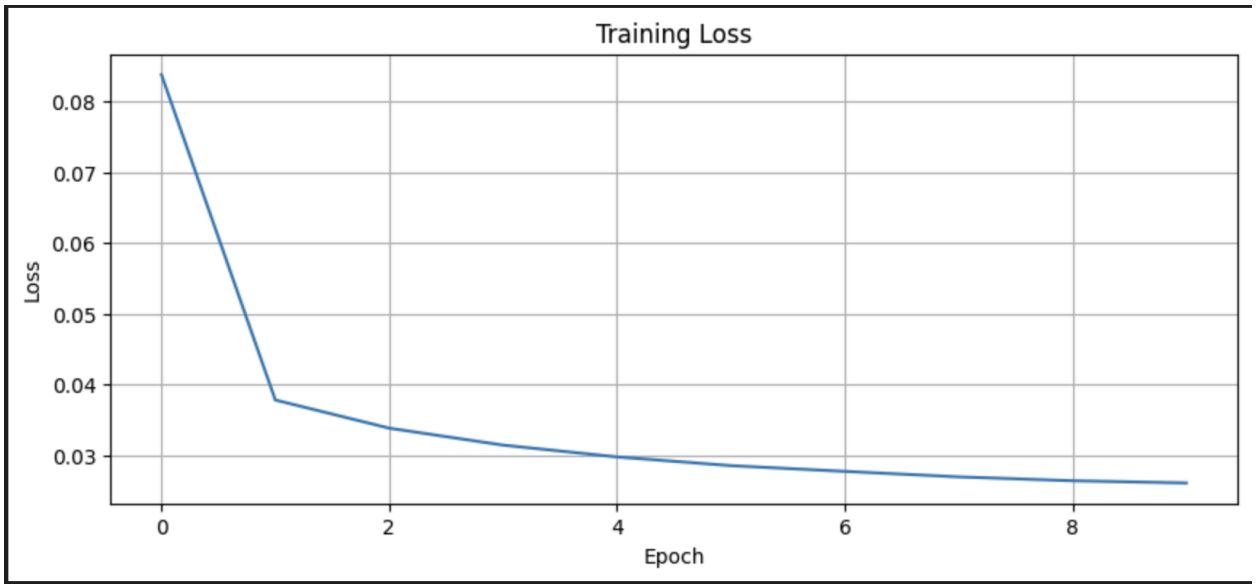
Time Embeddings: Because the U-Net needs to know how noisy the image is to predict the correct noise pattern, it uses Sinusoidal Positional Embeddings. This converts the current timestep t into a vector of sine and cosine waves, which is injected directly into the U-Net's convolutional blocks.

2. **Evaluation and Visualizations**

Training Loss Curve: The Mean Squared Error (MSE) loss steadily decreased over the 10 epochs as the U-Net improved at predicting the added Gaussian noise.

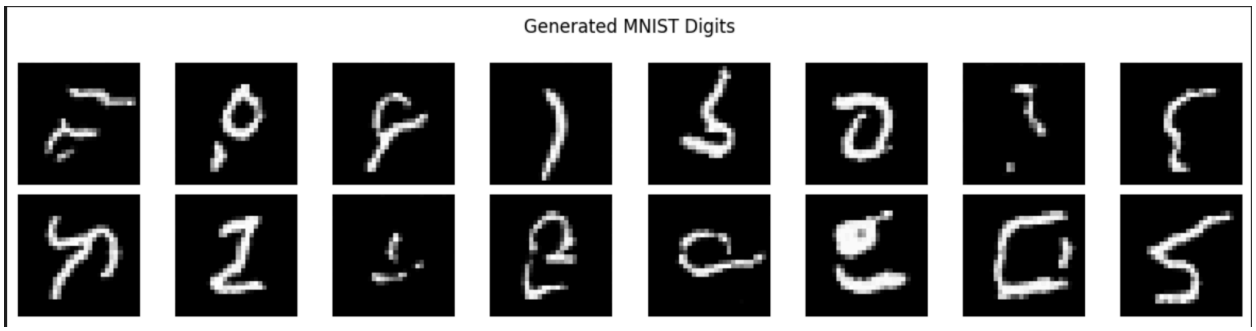
Generated MNIST Digits: The model successfully generated recognizable handwritten digits from pure random noise, alongside a step-by-step visualization of the denoising trajectory from t=1000 down to t=0.

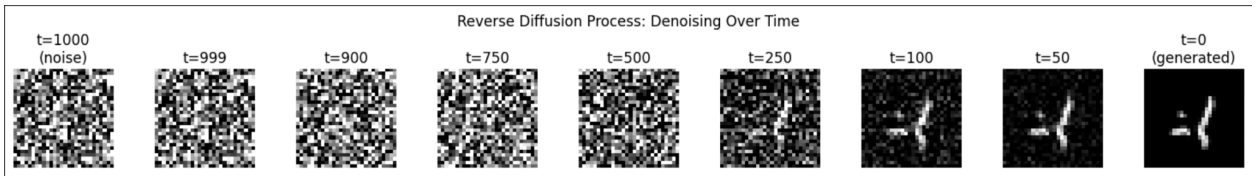
Training Loss Curve



The plot above demonstrates the Mean Squared Error (MSE) loss steadily decreasing over the 10 epochs as the U-Net gets better at predicting the added Gaussian noise.

Generated MNIST Digits





The outputs above show the model successfully generating recognizable handwritten digits from pure random noise, alongside a step-by-step visualization of the denoising trajectory from $t=1000$ down to $t=0$.

3. Reflections and Take-aways

A major takeaway was learning that diffusion models do not actually predict the final image; rather, they are trained solely to predict the noise added at a given timestep. Time Embeddings are crucial. Without them, the U-Net would treat a slightly blurry digit and pure television static exactly the same way, making the iterative reverse process impossible. Seeing pure static slowly transform into recognizable digits really demystified how models like Stable Diffusion operate.